

Claude Code Build Roadmap & Acceptance Criteria

GPS Implementation Specification Pack • Prepared for Claude Code • August 28, 2026 • Draft implementation directives; requires securities/compliance counsel review before live personalized recommendations or execution.

Implementation order

Phase	Deliverable	Acceptance criteria
0	Repository discovery and architecture map	Claude Code inventories existing tables, APIs, feature flags, auth/tiers, data feeds, jobs, and UI routes. No behavior changes yet.
1	Policy/config domain	Versioned policy tables/config: tier limits, risk bands, cooldown thresholds, universe criteria, strategy parameters. No hard-coded UI policy values.
2	Account and risk engine	Verified/estimated account status; sizing, allocation, correlation, daily/48h/30d metrics, circuit-breaker state machine; full tests.
3	Universe/data-quality engine	Eligibility filter, freshness checks, event gating, data provenance, fail-closed behavior.
4	Trend Pullback v1	Closed-bar deterministic scan, score explanation, entry/stop/target/expiry, audit records; paper-trade only.
5	Trade lifecycle	Plan states, TP/runner/Master-Profit floor model, user review screens, no blocked exits.
6	Tier UX/promotion	Entitlements, scan limits, readiness score, education flow; no upgrade bypasses.
7	Validation and monitoring	Backtest/shadow modules, metrics, dashboard, alerts, rollback controls.
8	Additional strategies/markets	Only after v1 validation gates; one module/market at a time.

Database/domain model additions

Entity	Purpose
policy_versions	Immutable policy configuration, effective dates, approvals, rollback.
strategy_versions	Rules, parameters, score schema, data dependencies, status.
instrument_eligibility_snapshots	Universe pass/fail and underlying market/event data.
signal_evaluations	Every scan result, criteria evidence, score, expiry, source timestamps.
trade_plans	Versioned plan coordinates, sizing, lifecycle state, strategy/policy links.
account_snapshots	Verified/estimated equity, buying power, holdings, data freshness.
risk_snapshots	Open risk, allocation, correlation, daily/48h/30d drawdown values.
cooldown_events	State transitions, trigger data, notification/acknowledgement, reset completion.
user_execution_reviews	Plan-adherence metrics and score inputs.
audit_events	Append-only record of system/user actions, decisions, and version references.

Backend enforcement requirements

- Enforce risk, tier, cooldown, and eligibility rules server-side. UI controls are not sufficient.
- Use closed-candle data only for confirmed signals.
- Fail closed for new-entry authorization when account data, market data, or event data is stale/unknown beyond defined policies.
- Implement idempotent jobs and unique event keys so repeated scans/price updates do not duplicate signals or alerts.

- Use transaction boundaries/locking when creating a plan or consuming a daily entry allowance.
- Add rate limits and authorization checks by tier and account/user.
- Return structured rejection reasons so UI can explain “why not” without exposing sensitive internals.
- Never send an order; initial release produces plans/alerts only unless separately authorized after legal/broker review.

Test matrix

Test area	Required cases
Sizing	Whole/fractional quantity, allocation cap, risk cap, insufficient funds, stop-distance zero/invalid
Cooldown	Every threshold crossing, recovery, deposits/withdrawals, realized/unrealized P&L, protective exit allowed
Signal	Closed-bar confirmation, stale data, event conflict, resistance-too-close, expired signal, invalidation
Lifecycle	TP1/TP2 ordering, gap through stop/floor, runner floor never decreases, partial-fill handling
Tier	Scan counts, manual scan limits, upgrade timing, no cooldown bypass
Audit	Every displayed plan can be reconstructed from persisted inputs/version IDs
Security	Row-level access, no cross-user account data, secrets server-only, rate-limit abuse tests

GitHub, Supabase, and Vercel workflow

- Create a dedicated feature branch; do not merge to main.
- Create migrations in small, reversible, reviewed units. Include rollback notes and seed/test fixtures.
- Use Supabase Row Level Security and server-side service roles only where necessary; test policies.
- Run unit, integration, migration, lint, type, and build checks before PR.
- Create a pull request with a clear change log, screenshots/preview evidence, migration list, test results, known limitations, and rollback plan.
- Create a Vercel preview deployment; do not deploy production or merge to main without explicit user approval.
- Maintain a single implementation tracker mapping every requirement in PDFs 01–07 to code location, test, and acceptance result.

Definition of done for v1

- A Novice user can see either one or more qualified paper-trade swing plans or a valid “No qualified setup” result.
- Every plan has a reproducible explanation, entry, stop, targets, expiry, sizing, account-risk result, and strategy/policy versions.
- Three-entry cap and all risk/cooldown states are enforced server-side, while exits remain unblocked.
- Small-account/fractional-share feasibility is surfaced before plan activation.
- No PSAR-confidential logic is present; all implementation is independently documented.
- No wording promises outcomes or represents score as probability of profit.
- System has auditability, test coverage, data freshness protection, and a reviewed preview/PR before any production release.